

EXAMEN DE PROGRAMACIÓN – 1º GRADO SUPERIOR

Examen 20 · Repaso integral de Java

Repaso de fundamentos, POO, colecciones y streams

Ficheros, excepciones y Optional

Hilos, JDBC y Swing a nivel básico

Ordenación, búsqueda y estructuras dinámicas

Asignatura	Programación (PRG)
Curso	1º Grado Superior — Desarrollo de Aplicaciones Web (DAW)
Duración	2 horas
Puntuación total	16 puntos
Convocatoria	Final
Valoración	Examen final de repaso

Parte 1: Opción múltiple y Verdadero/Falso

5 puntos — 0,5 cada pregunta

1. ¿Qué devuelve `"Hola".substring(1, 3)`?
- a) `"ol"`
 - b) `"ola"`
 - c) `"Ho"`
 - d) `"lo"`

RESPUESTA / SOLUCIÓN

a) `"ol"` — `substring(inicio, fin)` toma los caracteres desde el índice 1 (incluido) hasta el 3 (excluido): `'o'` y `'l'`.

-
2. ¿Qué ocurre al ejecutar `List.of(1, 2, 3).add(4)`?
- a) `UnsupportedOperationException`
 - b) Se añade el 4
 - c) `NullPointerException`
 - d) `ArrayIndexOutOfBoundsException`

RESPUESTA / SOLUCIÓN

a) `UnsupportedOperationException` — `List.of()` devuelve una lista inmutable; no permite `add/remove/set`.

-
3. ¿Cuál es el resultado de la expresión `5 % 2`?
- a) 1
 - b) 2

- c) 2.5
- d) 0

RESPUESTA / SOLUCIÓN

a) 1 — % es el operador módulo (resto de la división entera): 5 entre 2 da cociente 2 y resto 1.

4. ¿Qué clase se recomienda para concatenar muchas cadenas dentro de un bucle?

- a) StringBuilder
- b) String con +
- c) StringBuffer solo en un hilo
- d) char[]

RESPUESTA / SOLUCIÓN

a) StringBuilder — Cada + crea un String nuevo ($O(n^2)$ en bucles); StringBuilder muta el buffer en $O(1)$ amortizado por append.

5. En try-with-resources, si el cuerpo lanza una excepción y el cierre del recurso también, ¿qué ocurre?

- a) Se lanza la del cuerpo y la del cierre se añade como suprimida (addSuppressed)
- b) Se lanza la del cierre
- c) Se lanzan ambas como RuntimeException
- d) No se lanza ninguna

RESPUESTA / SOLUCIÓN

a) Se lanza la excepción del cuerpo y la del cierre se registra como suprimida — accesible con `getSuppressed()`.

6. Un método `static` puede invocarse sin crear una instancia de su clase.

RESPUESTA / SOLUCIÓN

Verdadero — Los miembros `static` pertenecen a la clase, no a los objetos; se invocan con `NombreClase.metodo()`.

7. ¿Cuánto suma `IntStream.range(1, 4).sum()`?

- a) 6
- b) 10
- c) 3
- d) 7

RESPUESTA / SOLUCIÓN

a) 6 — `range(1, 4)` genera 1, 2, 3 (el fin se excluye) y su suma es 6.

8. ¿Qué método de Map devuelve un valor por defecto si la clave no existe?

- a) `getOrDefault(clave, defecto)`
- b) `getOrNull(clave)`
- c) `get(clave)`
- d) `valueOrDefault(clave)`

RESPUESTA / SOLUCIÓN

a) `getOrDefault(clave, defecto)` — Devuelve el valor asociado o el defecto si la clave no está en el mapa.

9. ¿Qué excepción lanza `Integer.parseInt("abc")`?

- a) `NumberFormatException`
- b) `IOException`
- c) `ParseException`
- d) `ClassCastException`

RESPUESTA / SOLUCIÓN

a) `NumberFormatException` — Es una `RuntimeException` lanzada cuando la cadena no representa un entero válido.

10. Los records de Java generan automáticamente `equals()`, `hashCode()` y `toString()` basados en sus componentes.

RESPUESTA / SOLUCIÓN

Verdadero — Un record declara los campos en el encabezado y el compilador genera el constructor canónico, acceso, equals, hashCode y toString.

Parte 2: Análisis y depuración

3 puntos

1. El siguiente código calcula un valor por defecto pero se ejecuta más de lo necesario. ¿Por qué? ¿Cómo se corrige? (1,5 pts)

```
import java.util.Optional;\npublic class Defecto {\n    static String caro() {\n
```

```
System.out.println("Calculando...");\n    }\n}
```

RESPUESTA / SOLUCIÓN

`orElse(valor)` evalúa el argumento SIEMPRE, aunque el `Optional` tenga valor: se imprime 'Calculando...' sin necesidad.

El coste es doble: cálculo inútil (y efectos secundarios visibles).

Corrección: `orElseGet(() -> caro())` — el `Supplier` solo se ejecuta cuando el `Optional` está vacío:

```
System.out.println(opt.orElseGet(Defecto::caro)); // no imprime 'Calculando...'
```

2. Este código lanza `ConcurrentModificationException`. ¿Por qué y cómo se arregla? (1,5 pts)

```
import java.util.*;\npublic class FiltroLista {\n    public static void main(String[] args) {\n
```

RESPUESTA / SOLUCIÓN

El `for-each` usa un `Iterator` interno; `nums.remove()` modifica estructuralmente la lista mientras se itera.

El iterador detecta la modificación y lanza `ConcurrentModificationException`.

Soluciones:

1) `Iterator` explícito:

```
Iterator<Integer> it = nums.iterator();
```

```
while (it.hasNext()) { if (it.next() % 2 == 0) it.remove(); }
```

2) O filtrar con streams: `nums.removeIf(n -> n % 2 == 0);`

Parte 3: Ejercicios de programación

6 puntos

1. Usando streams, escribe un método que reciba una lista de palabras y devuelva la palabra más larga (o cadena vacía si la lista es vacía). (2 pts)

RESPUESTA / SOLUCIÓN

```
import java.util.*;

public class PalabraMasLarga {

    static String masLarga(List<String> palabras) {
        return palabras.stream()
            .max(Comparator.comparingInt(String::length))
            .orElse("");
    }

    public static void main(String[] args) {
        List<String> lista = List.of("java", "programacion", "hola", "a");
        System.out.println(masLarga(lista)); // programacion
    }
}
```

-
2. Escribe un programa que lea un fichero numeros.txt (un entero por línea) y muestre la suma y el máximo usando Files.lines y streams. (2 pts)

RESPUESTA / SOLUCIÓN

```
import java.nio.file.*;
import java.io.IOException;

public class SumaFichero {

    public static void main(String[] args) {

        try (var lineas = Files.lines(Paths.get("numeros.txt"))) {

            var nums = lineas.filter(l -> !l.isBlank()).mapToInt(Integer::parseInt);

            System.out.println("Suma: " + nums.sum());

        } catch (IOException e) {

            e.printStackTrace();

        }

    }

}
```

Nota: el stream de Files.lines se cierra con try-with-resources.

-
- 3. Define la clase Libro (titulo, autor, paginas) con constructor y getters, y usando streams: ordena una lista por páginas ascendente y muestra solo los libros de más de 300 páginas. (2 ptos)**

RESPUESTA / SOLUCIÓN

```
import java.util.*;

public class Libro {

    private final String titulo;
    private final String autor;
    private final int paginas;

    public Libro(String titulo, String autor, int paginas) {
        this.titulo = titulo; this.autor = autor; this.paginas = paginas;
    }

    public String getTitulo() { return titulo; }
    public int getPaginas() { return paginas; }
    public String toString() { return titulo + " (" + paginas + "p)"; }
    public static void main(String[] args) {

        List<Libro> libros = List.of(
            new Libro("A", "X", 450),
            new Libro("B", "Y", 250),
            new Libro("C", "Z", 600));

        libros.stream()
            .filter(l -> l.getPaginas() > 300)
            .sorted(Comparator.comparingInt(Libro::getPaginas))
            .forEach(System.out::println);

        // A (450p), C (600p)
    }
}
```

Parte 4: Pregunta teórica

2 puntos

1. Compara `Optional.orElse(valor)` y `Optional.orElseGet(supplier)`. ¿Cuándo usarías cada uno? Pon un ejemplo.

RESPUESTA / SOLUCIÓN

`orElse(valor)` recibe el valor ya calculado y lo evalúa SIEMPRE, aunque el `Optional` no esté vacío.

`orElseGet(supplier)` recibe una función que solo se ejecuta si el `Optional` está vacío.

Usar `orElseGet` cuando el valor por defecto es caro de calcular o tiene efectos secundarios (consultas, logs, creación de objetos pesados).

Ejemplo: `opt.orElseGet(() -> cargarDeBD())` evita la consulta cuando el valor ya existe;

`opt.orElse("desconocido")` es correcto para constantes baratas.

Plantilla de respuestas

(Páginas en blanco para desarrollar las soluciones)